REP-1: Inventory Report Upgrade

Instructions for Mo (Part 4, back office and inventory)

Prepared 2026-11-04 from the TA's Checkpoint 3 feedback and the current code. Size L. Owner: Mo.

Item	Details
Your report	Inventory report, page **/staff/reports/inventory**. Today it lists stock against the reorder level, stock value and supplier orders by status.
What you add	Inventory turnover, fast and slow movers, products unsold for a long time, estimated days until stock runs out, supplier delivery performance, a reorder suggestion, filters, KPI cards with changes, charts and a 'What this means' panel.
Files you create	`db/reports/inventory_analysis.sql`, `db/reports/test_inventory_analysis.sql`, `lib/inventoryMetrics.js`, `lib/supplierMetrics.js`, `server/reports/suppliers.js`, tests under `tests/`.
Files you change	`server/reports/inventory.js`, `src/app/pages/reports/InventoryReportPage.jsx` (and a small folder of local components next to it).
Files you must not touch	`db/core/schema.sql` (frozen), `db/core/views.sql` (new views go in your own file), `package.json` (REP-7 installs the chart library).
Depends on	Nothing to start. It looks best once REP-0 (demo data) and DEL-3 (suppliers with lead times) are merged. Until then, test with your own small inserts inside a rolled-back script.
Done when	A product with no recent sales shows as Dead, a fast seller with low stock shows days left under its lead time, and the supplier table shows at least one late supplier on the demo data.

Important. SQL in this guide is a hand-checked starting point. It was written against `schema.sql` but could not be run here (there is no SQL Server in the assistant's workspace). Run it on your local SQL Server Express first. Never run anything against the hosted Azure database yourself: the lead loads new SQL there (INT-1).

1. Why this report has to change

The TA's overall comment on all reports was: *"Your reports should go beyond showing what happened and help management understand why it happened, whether performance is improving or declining, and what actions the business should take. Include more meaningful calculations, comparisons, trends, visuals and flexible filters."*

For the Inventory report the TA asked for inventory turnover, fast and slow movers, products unsold for long periods, an estimate of days until stock runs out based on sales trends, and supplier performance (delivery times and delays), so management can decide **what to reorder, what is overstocked and where costs can be reduced**.

What exists today versus what the TA asked for

TA expectation	Today	What you add
Inventory turnover	Not shown. Only value at cost.	Turnover per year = COGS of the window, scaled to a year, divided by stock value at cost. Shown as a KPI with change against the previous 30 days.
Fast and slow movers	Not shown. The table is sorted by distance from the reorder level.	Every product gets a movement class: Fast, Normal, Slow or Dead. Filter and chart by class.
Products unsold for long periods	Not shown.	Last sale date, days since last sale, 'Dead' class (no sale in 90 days) and the cash tied up in them.
Days until stock runs out, from sales trends	Only a static reorder level.	Days of stock left = stock / daily sales rate. Compared with the supplier's lead time. A trend column (last 30 days against the 30 before).
Supplier performance	Only a count of orders by status.	New supplier panel: average days to deliver, on-time percentage, average days late, cancelled percentage, spend, overdue open orders.
What to reorder, what is overstocked, where to cut cost	Management must work it out by eye.	A suggested reorder quantity per product, an overstock list and a 'What this means' panel with rule-based sentences and actions.
Flexible filters, comparisons, visuals	None. One table and four status counts.	Filters (below), KPI cards with changes, four charts, sortable table.

The point of the change: today a manager sees *what is in stock*. After the change a manager sees *what will go wrong, what is wasted, and what to do about it*. Every number you add should answer one of those three questions, and every chart should have a sentence next to it saying what to do.

2. The data you will use

Table	Columns you need	Used for
PRODUCT	Product_id, SKU, Name, Cost_price, Sell_price, Stock_quantity, Reorder_threshold, Category_id, Is_active	Stock, stock value at cost, reorder level.
CATEGORY	Category_id, Name	Category filter and grouping.
SALES_TRANSACTION	Transaction_id, Transaction_date, Status	Only Status = 'completed' counts as a sale.
TRANSACTION_ITEM	Transaction_id, Product_id, Quantity, Line_total	Units sold per product and period.
SUPPLIER	Supplier_id, Supplier_name	Supplier filter and names.
SUPPLIER_PRODUCT	Supplier_id, Product_id, Unit_cost, Lead_time_days	Promised delivery time. Lead time is stored per supplier and product, not per order.
SUPPLIER_ORDER	Order_id, Supplier_id, Order_date, Received_date, Status (Pending, Ordered, Received, Cancelled)	Actual delivery time and delays.
SUPPLIER_ORDER_ITEM	Order_id, Product_id, Quantity, Unit_cost	Units on order, order cost.
LOW_STOCK_ALERT	Product_id, Triggered_at, Resolved_at, Order_id	Already used by the current view for alert age.

> **Rule.** Every query and every join must filter **Marked_for_deletion = 0**, including the tables you only join (CATEGORY, SUPPLIER, SUPPLIER_PRODUCT, order items). The application login cannot DELETE rows; the project uses soft delete everywhere.

Limits you must respect and print on the page

- **No stock history.** The database stores today's stock only. A KPI like inventory value cannot be compared with last month. Show the value without a change and say so ('stock history is not stored'). Sales-based KPIs (turnover, units sold) can be compared with the previous 30 days.
- **COGS uses today's cost.** `TRANSACTION_ITEM` has no cost column. COGS = units sold times `PRODUCT.Cost_price` now. Label it 'COGS at current cost'.
- **Turnover is approximate.** It divides by today's stock value instead of the average stock during the period. Label it 'approximate'.
- **Units sold are gross.** Refunded units are not subtracted. If you want net demand, subtract units from refunded returns, but state it on the page.
- **'Never sold' means no completed sale on record.** Products have no created date, so a brand-new product also looks unsold. Show 'No sales on record' instead of 'Dead' when the product has a stock of zero and no history.
- **Seed data is tiny** (8 sales, no suppliers, no orders). Every panel needs an empty state that explains what is missing instead of showing zeros.

3. New parameters (filters)

The page keeps working with no parameters at all. Every parameter is optional, validated on the server, and reflected in the URL so a filtered view can be shared or reloaded.

3.1 Inventory endpoint: GET /api/reports/inventory

Parameter	Values	Default	Effect and validation
category	A Category_id (integer)	all	Only products of that category. Reject non-integers with HTTP 400.
cls	fast, normal, slow, dead	all	Only products in that movement class. Anything else: 400. The class is computed in the library, so apply this filter after the class is known (in JavaScript).
level	ok, low, out	all	Matches the existing Stock_status of the view ('OK', 'Low', 'Out of stock').
supplier	A Supplier_id (integer)	all	Only products that supplier carries. Use EXISTS on SUPPLIER_PRODUCT with Marked_for_deletion = 0.
window	30, 60, 90 (days)	30	Which sales window sets the daily sales rate for days-left and reorder. The trend always compares last 30 days with the 30 before.
risk	1 (only)	off	Only products whose days of stock left is below their lead time plus the safety margin (the 'will run out' list).
inactive	1 (include)	off	Include paused products (Is_active = 0). Today's report lists active products only; paused products still hold stock, so they count toward value when included.
sort	days_left, value, units_30, last_sale, name	days_left	Sort of the detail table. Whitelist the column names; never paste user text into the SQL.
search	Text	none	Name or SKU contains the text. Escape % and _ the way `server/manager/product-catalog.js` does.

3.2 Supplier endpoint: GET /api/reports/suppliers (new, manager only)

Parameter	Values	Default	Effect and validation
from, to	YYYY-MM-DD	last 180 days	Orders whose Order_date is in the range. Reuse the real-calendar-date check from `server/reports/sales.js`; start after end is 400.
supplier	A Supplier_id	all	One supplier.
status	Pending, Ordered, Received, Cancelled	all	Orders in that status.
compare	1	on	Also compute the same metrics for the period of equal length just before `from` (local helper until REP-7 provides the shared one).

Design choice. Why a separate supplier endpoint: the TA found that Support users can read supplier data in the inventory report. Spend and delivery data go in a manager-only endpoint now, so the page is correct whatever ACC-1 decides for the rest of the inventory report. The supplier panel in the page simply shows 'Managers only' when the call returns 403.

4. Metrics and formulas

Keep all calculations in pure functions in `lib/inventoryMetrics.js` and `lib/supplierMetrics.js` (not in `server/`, because every file in `server/` becomes a Vercel route). Each function takes plain rows and returns plain values, so it can be tested with fixtures and no database.

Metric	Formula	Notes
Daily sales rate	units sold in the window / window days	Window = 30, 60 or 90 from the `window` parameter. Rate 0 means no sales.
Days of stock left	Stock_quantity / daily rate	Return null when the rate is 0 (show 'no sales', never 'infinite'). Round down to whole days.
Lead time	MAX(SUPPLIER_PRODUCT.Lead_time_days) for the product	If no supplier is set, assume 7 days and mark the value 'assumed'. Keep the constant DEFAULT_LEAD_DAYS in the library.
Run-out risk	days left < lead time + SAFETY_DAYS (3)	The product will be empty before a new order could arrive.
Trend	(Units_30 - Units_prev_30) / Units_prev_30	Null when Units_prev_30 is 0. 'Rising' at +25% or more, 'Falling' at -25% or less.
COGS (window)	units in window x Cost_price	At current cost.
Turnover per year	COGS_window x (365 / window) / Stock_value_at_cost	Null when stock value is 0. Approximate, see limits.
Movement class	Dead: stock > 0 and Units_90 = 0. Of the products that did sell in 90 days: Fast = top 20% by Units_90, Slow = bottom 30%, Normal = the rest.	Percent-based, so it works with any amount of data. Put the cut-offs in constants. A product with stock 0 and sales is classed by its sales.
Overstock	days of stock left > 180 and the product sold in 90 days	Overstock cash = Stock_value_at_cost of those products.
Dead stock value	SUM(Stock_value_at_cost) of Dead products	Cash tied up in products nobody bought in 90 days.
Suggested reorder quantity	CEILING(rate x (lead time + 14) - stock - On_order_qty), never below 0	14 is the cover period after delivery (constant COVER_DAYS). Subtracting On_order_qty (open Pending or Ordered quantities) avoids ordering twice.
Days to deliver (order)	Received_date - Order_date	Only for orders with a Received_date.
Days late (order)	Days to deliver - promised lead time	Zero or negative = on time or early. Null if the promised time is unknown.
On-time percent (supplier)	received orders with days late <= 0 / received orders	Show 'not enough data' under 3 received orders.
Cancelled percent	cancelled orders / all orders of the supplier	In the date range.
Overdue open order	Status Pending or Ordered and days open > promised lead time	Counted in a KPI and listed by name.

5. Build procedure

Do the steps in order. Each ends with something you can check before moving on.

Step 1. Create the two views (db/reports/inventory_analysis.sql)

A view cannot take parameters, so the views hold the heavy joins and the endpoint adds the filters. Use `CREATE OR ALTER VIEW` so the file can be run again.

```
-- VW_INVENTORY_ANALYSIS: one row per product (not flagged), sales windows included
CREATE OR ALTER VIEW VW_INVENTORY_ANALYSIS AS
SELECT p.Product_id, p.SKU, p.Name AS Product_name, c.Name AS Category_name, p.Category_id,
       p.Is_active, p.Cost_price, p.Sell_price, p.Stock_quantity, p.Reorder_threshold,
       CAST(p.Stock_quantity * p.Cost_price AS DECIMAL(12,2)) AS Stock_value_at_cost,
       ISNULL(sales.Units_30, 0)      AS Units_30,
       ISNULL(sales.Units_60, 0)      AS Units_60,
       ISNULL(sales.Units_90, 0)      AS Units_90,
       ISNULL(sales.Units_prev_30, 0) AS Units_prev_30,
       sales.Last_sale_date,
       DATEDIFF(DAY, sales.Last_sale_date, d.Today) AS Days_since_last_sale,
       ISNULL(oo.On_order_qty, 0)     AS On_order_qty,
       sp.Lead_time_days, sp.Supplier_count
FROM   PRODUCT p
JOIN   CATEGORY c ON c.Category_id = p.Category_id AND c.Marked_for_deletion = 0
CROSS APPLY (SELECT CAST(SYSDATETIME() AS DATE) AS Today) d
OUTER APPLY (
    SELECT SUM(IIF(q.D >  DATEADD(DAY, -30, d.Today), q.Qty, 0)) AS Units_30,
           SUM(IIF(q.D >  DATEADD(DAY, -60, d.Today), q.Qty, 0)) AS Units_60,
           SUM(IIF(q.D >  DATEADD(DAY, -90, d.Today), q.Qty, 0)) AS Units_90,
           SUM(IIF(q.D >  DATEADD(DAY, -60, d.Today)
               AND q.D <= DATEADD(DAY, -30, d.Today), q.Qty, 0)) AS Units_prev_30,
           MAX(q.D) AS Last_sale_date
    FROM (SELECT CAST(s.Transaction_date AS DATE) AS D, ti.Quantity AS Qty
          FROM   TRANSACTION_ITEM ti
          JOIN   SALES_TRANSACTION s ON s.Transaction_id = ti.Transaction_id
          WHERE  ti.Product_id = p.Product_id
            AND  ti.Marked_for_deletion = 0
            AND  s.Status = 'completed' AND s.Marked_for_deletion = 0) q
) sales
OUTER APPLY (
    SELECT MAX(x.Lead_time_days) AS Lead_time_days, COUNT(*) AS Supplier_count
    FROM   SUPPLIER_PRODUCT x
    JOIN   SUPPLIER su ON su.Supplier_id = x.Supplier_id AND su.Marked_for_deletion = 0
    WHERE  x.Product_id = p.Product_id AND x.Marked_for_deletion = 0
) sp
OUTER APPLY (
    SELECT SUM(oi.Quantity) AS On_order_qty
    FROM   SUPPLIER_ORDER_ITEM oi
    JOIN   SUPPLIER_ORDER o ON o.Order_id = oi.Order_id
    WHERE  oi.Product_id = p.Product_id AND oi.Marked_for_deletion = 0
      AND  o.Status IN ('Pending', 'Ordered') AND o.Marked_for_deletion = 0
) oo
WHERE  p.Marked_for_deletion = 0;
GO

-- VW_SUPPLIER_DELIVERIES: one row per supplier order (not flagged)
CREATE OR ALTER VIEW VW_SUPPLIER_DELIVERIES AS
SELECT o.Order_id, o.Supplier_id, su.Supplier_name, o.Order_date, o.Received_date, o.Status,
       it.Items, it.Units, CAST(it.Cost AS DECIMAL(12,2)) AS Order_cost,
       lt.Promised_days,
       DATEDIFF(DAY, o.Order_date, o.Received_date) AS Days_to_deliver,
       CASE WHEN o.Received_date IS NULL OR lt.Promised_days IS NULL THEN NULL
            ELSE DATEDIFF(DAY, o.Order_date, o.Received_date) - lt.Promised_days END AS Days_late,
       CASE WHEN o.Received_date IS NULL AND o.Status IN ('Pending', 'Ordered')
            THEN DATEDIFF(DAY, o.Order_date, CAST(SYSDATETIME() AS DATE)) END AS Days_open
FROM   SUPPLIER_ORDER o
JOIN   SUPPLIER su ON su.Supplier_id = o.Supplier_id AND su.Marked_for_deletion = 0
OUTER APPLY (SELECT COUNT(*) AS Items, SUM(oi.Quantity) AS Units,
```

```
                SUM(oi.Quantity * oi.Unit_cost) AS Cost
        FROM    SUPPLIER_ORDER_ITEM oi
        WHERE   oi.Order_id = o.Order_id AND oi.Marked_for_deletion = 0) it
OUTER APPLY (SELECT MAX(sp.Lead_time_days) AS Promised_days
        FROM    SUPPLIER_ORDER_ITEM oi
        JOIN    SUPPLIER_PRODUCT sp ON sp.Supplier_id = o.Supplier_id
                                    AND sp.Product_id  = oi.Product_id
                                    AND sp.Marked_for_deletion = 0
        WHERE   oi.Order_id = o.Order_id AND oi.Marked_for_deletion = 0) lt
WHERE  o.Marked_for_deletion = 0;
GO
```

Check: run the file twice (it must not fail the second time) and run `SELECT TOP 5 * FROM VW_INVENTORY_ANALYSIS ORDER BY Units_90 DESC`. Compare one product by hand against `TRANSACTION_ITEM`.

Step 2. Write a rolled-back test script (db/reports/test_inventory_analysis.sql)

Follow the pattern of `verify.sql`: open a transaction, insert known rows, assert the view values, then ROLLBACK so nothing is left behind. Cover at least:

- A product sold 10 units 5 days ago and 4 units 40 days ago: Units_30 = 10, Units_prev_30 = 4, Units_90 = 14, Last_sale_date correct.
- A product with stock and no sale: Units_90 = 0 and Last_sale_date is NULL.
- A sale with Status 'open' or a flagged line: not counted.
- A supplier order of 3 items with different lead times: Promised_days is the maximum; received 9 days after ordering with promised 7 gives Days_late = 2.
- A flagged supplier or flagged order item: not counted.

Step 3. Write the pure metric modules (lib/inventoryMetrics.js and lib/supplierMetrics.js)

Suggested exports (names are suggestions, keep them consistent in tests):

Function	Input	Output
buildProductRows(rows, windowDays)	Rows from VW_INVENTORY_ANALYSIS	Same rows plus dailyRate, daysLeft, leadDays, leadAssumed, trend, cogsWindow, atRisk, suggestedReorder.
classify(rows)	Rows from the step above	Adds cls: 'fast', 'normal', 'slow' or 'dead'.
summarize(rows, windowDays)	Classified rows	KPIs: inventoryValue, turnover, turnoverPrev, toReorder, atRiskCount, overstockCash, deadStockValue, valueByClass.
buildInventoryInsights(rows, supplierSummary)	Classified rows, optional supplier summary	Up to 5 objects { id, severity, text, action }, most severe first. See section 7.
summarizeSuppliers(orders, today)	Rows from VW_SUPPLIER_DELIVERIES	Per supplier: orders, received, avgDaysToDeliver, onTimePct, avgDaysLate, cancelledPct, spend, overdueOpen; plus monthly late series.

Use integers for cents when summing money, the way `inventory.js` already does for the value total.

Step 4. Extend the endpoints

1. In `server/reports/inventory.js` keep the existing keys **products, ordersByStatus, totals** exactly as they are, so nothing that reads them breaks. Add new keys: **kpis, classes, movers, insights, supplierSummary (small), filters** (echo of the accepted parameters).
2. Add a second query on `VW_INVENTORY_ANALYSIS` using the filters that can be applied in SQL (category, search, inactive, supplier via EXISTS). Pass values as named parameters, never by pasting text into the SQL.
3. Compute class, days left and the rest with the library, then apply `cls`, `level` and `risk` in JavaScript, then sort.
4. Create `server/reports/suppliers.js` with `requireAuth(req, ['manager'])` and the parameters of section 3.2.
5. Run **npm run routes** so `lib/routes.generated.js` includes the new endpoint, and commit it. `npm test` fails if it is stale.

> **Do not.** Keep the Support-or-manager rule on the inventory endpoint as it is today. ACC-1 will change who can open it; do not mix that change into this work.

Step 5. Rebuild the page (InventoryReportPage.jsx)

1. Put new small components in `src/app/pages/reports/inventory/` (filter bar, KPI card, mover table, supplier panel, insights list). When REP-7 merges you replace them with the shared ones; keeping them local avoids merge conflicts now.
2. Layout, top to bottom: page header, filter bar, KPI cards, 'What this means' panel, charts, product table, supplier panel. The wireframe on the last page shows it.
3. Read filters from the URL query string and write them back when they change.
4. Charts: until REP-7 adds a chart library, draw bars with plain `div` elements whose width is set by inline style (inline styles are allowed by the site's security policy; external chart scripts are not). Swap to the shared chart components afterwards.
5. Every KPI shows its value, its change in words as well as colour ('up 12% on the previous 30 days'), and a one-line meaning.
6. Table: sortable, with a text label next to every colour (Dead, Slow, Fast, 'Runs out in 6 days'). A manager must be able to read the page without relying on colour.
7. Empty states: no products match the filters; no supplier orders yet; no sales in the window.

Step 6. Tests

Test file	What it checks	How
tests/lib/inventoryMetrics.test.js	Class boundaries, days left with rate 0, reorder quantity rounding and the on-order subtraction, run-out risk, insight rules and their ordering.	Plain fixtures, no database.
tests/lib/supplierMetrics.test.js	On-time percent, days late (early, on time, late), cancelled percent, overdue open orders, fewer than 3 orders gives 'not enough data'.	Plain fixtures.
tests/inventoryReport.test.js	Parameters are validated (bad category gives 400), valid filters reach the query as named parameters, old keys are still returned, support still gets 200, no login gives 401.	Use `tests/helpers_manager.js` with `jest.mock('../lib/db', ...)` as in `tests/catalog.test.js`.
tests/suppliersReport.test.js	Manager gets 200, support gets 403, no login 401, bad dates 400.	Same helpers.

Do not use `helpers_backoffice.js` for filter tests: its request builder does not support query parameters. It is fine for a smoke test against the real local database.

6. Visual KPIs and chart ideas

Order matters: the reader first sees the six cards, then the meaning panel, then the evidence in charts. Each item below says what question it answers, so you can cut anything that does not answer one.

6.1 KPI cards

Card	Value	Change shown	Question it answers
Inventory value	SUM of stock x cost	None (no stock history). Show the number of products next to it.	How much money is sitting on the shelves?
Turnover (per year)	Formula in section 4	Against the previous 30 days.	Are we selling stock fast enough for the money invested?
To reorder now	Count of products at run-out risk	How many are out of stock right now.	What must be ordered today?
Overstock cash	Value of overstocked products	Number of products.	Where is cash wasted on too much stock?
Dead stock	Value of Dead products	Number of products and oldest last-sale date.	What has not sold for 90 days?
Supplier on-time	On-time percent, last 180 days	Points up or down against the 180 days before.	Can we trust our suppliers' lead times?

6.2 Charts

Chart	Type	Data	What it shows
Stock value by movement class	Stacked bar or donut	valueByClass	How much money is in Fast, Normal, Slow and Dead stock. A big Dead slice is the cost-reduction story.
Days of stock left against lead time	Horizontal bars with a marker line per product (top 10 most urgent)	daysLeft, leadDays	Bars shorter than the marker (in red) will be empty before a delivery can arrive. This is the reorder list as a picture.
Sales rate against stock on hand	Scatter, one dot per product, rate on one axis and stock on the other	dailyRate, stock	Four quadrants: fast and low stock (reorder), fast and high (healthy), slow and high (overstock), slow and low (fine). Hover shows the product.
Supplier on-time percent	Bars, one per supplier, with the target line at 90%	summarizeSuppliers	Which supplier to pressure or replace.
Late deliveries by month	Line, with the previous period dashed	monthly late series	Whether delivery performance is improving or declining (the TA's wording).
Top 10 dead stock by value	Horizontal bars	Dead rows by value	Exactly which products to discount or return.
Trend sparkline per product	Tiny bars in the table: Units_prev_30 and Units_30	both columns	Rising or falling demand without opening anything.

Minimum for 'done': the first two charts and the supplier on-time bars. The others are extra credit if time allows.

7. 'What this means' panel: the insight rules

These are rule-based sentences, not guesses. Each rule has a condition, a sentence built from real numbers and a recommended action. Show at most five, most severe first. Put the rules in `buildInventoryInsights` so they can be tested.

Id	Severity	Condition	Sentence and action
runout	Danger	N products with days left below lead time + 3	"N products will run out before their supplier can deliver." Action: reorder now, open the list (sets the risk filter).
stockout	Danger	Products with stock 0 and Units_90 > 0	"N products that sell are out of stock; about $X of sales per day is being lost." (rate x Sell_price). Action: order immediately.
dead	Warning	Dead stock value > 0	"$X is tied up in N products with no sale in 90 days." Action: discount, bundle or return to the supplier.
overstock	Warning	Overstock cash > 0	"$X is tied up in N products with more than 180 days of stock." Action: pause reorders for these.
rising	Info	Products with Units_30 at least 1.5 x Units_prev_30 and Units_30 at least 5	"Demand rose 50% or more for N products." Action: raise their reorder level.
threshold	Info	Reorder_threshold below rate x lead time	"The reorder level of N products is too low for their sales speed." Action: raise it to the suggested number.

Id	Severity	Condition	Sentence and action
late	Warning	A supplier with on-time percent below 80% and at least 3 received orders	"Supplier X delivered late N of M times, on average D days late." Action: renegotiate or add a second supplier.
cancelled	Warning	A supplier with cancelled percent above 20%	"Supplier X cancelled P% of orders." Action: confirm stock before ordering.
overdue	Danger	Open orders older than their promised lead time	"N open orders are overdue (oldest: Supplier X, D days)." Action: contact the supplier.
allgood	Good	No danger or warning above	"No product is expected to run out before its next delivery."

8. Traps to avoid

- **Division by zero.** Rate 0 means no sales, not infinite days. Return null and print 'no sales'.
- **Soft delete.** A forgotten `Marked_for_deletion = 0` on a joined table makes deleted suppliers or order lines count. Test it.
- **Inactive products.** They are hidden by default but hold stock. The 'include paused' filter must change both the table and the KPIs.
- **Double counting lead times.** Lead time is per supplier and product. For a product with several suppliers use the maximum (conservative) and say so in a tooltip.
- **Ordering twice.** Subtract quantities already on open orders before suggesting a reorder.
- **Filtering after truncation.** Apply class and risk filters before any 'top 10' cut, not after.
- **Colour alone.** Always pair red or green with words.
- **Server files become routes.** Put helper code in `lib/`, not in `server/`.
- **Date handling.** The server's date is used for 'today'. On Azure this is UTC, so late in the evening local time the 30-day window can shift by one day. Acceptable; note it in the PR.

9. Definition of done

[ ]	Both views are in `db/reports/inventory_analysis.sql`, run twice without errors on a local database, and every join filters Marked_for_deletion = 0.
[ ]	The rolled-back test script passes on a local database and leaves no rows behind.
[ ]	Pure functions in `lib/` have tests for every formula in section 4, including zero rates and fewer than 3 orders.
[ ]	The inventory endpoint still returns products, ordersByStatus and totals unchanged, plus the new keys. Every parameter is validated and tested.
[ ]	The supplier endpoint is manager-only and tested (200, 403, 401, 400).
[ ]	`npm run routes` was run and `lib/routes.generated.js` is committed; `npm test` passes.
[ ]	The page has: filter bar, six KPI cards, meaning panel with 3 to 5 insights, at least three charts, sortable table, supplier panel, empty states.
[ ]	A product with no recent sales shows as Dead; a fast seller with low stock shows days left under its lead time; the demo data shows at least one late supplier.
[ ]	The page states its limits: COGS at current cost, turnover approximate, no stock history, gross units.

[ ]	The PR description lists the new parameters, the new files, and the manual checks you did (one product's numbers verified by hand).

Questions to ask the lead before you start: (1) Should paused products count in the KPIs by default? Suggested: no. (2) Is a 90-day Dead threshold right for the store? Suggested: yes, keep it as a constant. (3) Which supplier on-time target does the team want? Suggested: 90%.

Wireframe: how the upgraded Inventory report page could look

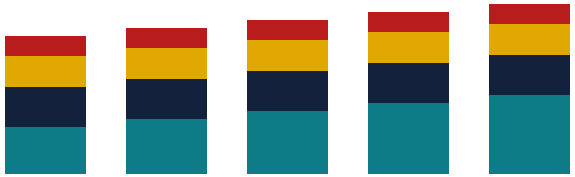
Inventory report

Stock health, movement and supplier performance (window: last 30 days)

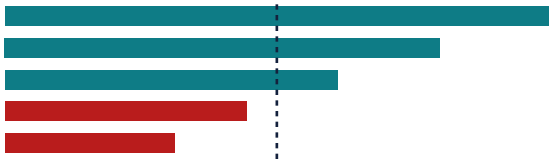
Category: All | Movement: All | Supplier: All | Stock level: All | Window: 30 days | Include paused: No | Search product or SKU

$48,200	3.4x	12	$6,900	$3,100	72%
Inventory value	Turnover (per year)	To reorder now	Overstock cash	Dead stock	Supplier on-time
no stock history stored	up 0.4 on previous 30 days	5 will run out first	14 products	9 products, 90 days idle	down 8 pts on prior 180d

Stock value by movement class
Fast, Normal, Slow, Dead



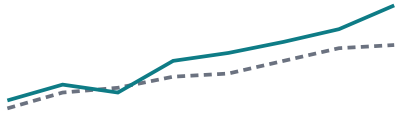
Days of stock left vs lead time
top 10 urgent; dashed = lead time



Sales rate vs stock on hand
one dot per product

Supplier on-time %
target line at 90%

Late deliveries by month
dashed = previous period



What this means

- 5 products will run out before their supplier can deliver.
  Reorder now: open the list.
- $3,100 is tied up in 9 products with no sale in 90 days.
  Discount, bundle or return them.
- Supplier B was late on 4 of 6 orders, 3 days late on average.
  Renegotiate or add a second supplier.
- Demand rose 50%+ for 3 products.
  Raise their reorder levels.
- 2 open orders are overdue.
  Contact the supplier.

Product	Category	Stock	Sold 30d	Trend	Days left	Lead time	Class	Reorder qty

Reading order: filters, six KPI cards, the 'What this means' panel (right), charts (left), then the sortable table. The numbers are examples only. Every chart box has a one-line caption that says what to look for.

Colour is never alone: red items also carry words (Dead, Runs out in 6 days). Until REP-7 installs a chart library, draw the bars with plain div elements and inline styles.